AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 10

Design Problem: A Multi-Agent Research System

The exam's Scenario 3: a coordinator with search, analysis, synthesis and report subagents. Each one can work, and the report can still be wrong: narrow in scope, stripped of sources, or quietly missing what a failed search should have found.

Module	AI06 of 13
Lesson	10 of 10, design problem
Prerequisites	AI06 L1–L8, AI05 L4, AI05 L6
Exam relevance	**Direct.** Scenario 3, combining TS 1.2, 1.3, 2.1, 2.2, 2.3, 5.3, 5.6
Reading time	~60 minutes

Contents

1. Design a Multi-Agent Research System
2. Module Revision

Lesson 10 of 10 · Agents and MCP

Design a Multi-Agent Research System

Key idea

The product isn't the search. It's a report whose every claim can be traced, whose gaps are stated, and whose cost matched the question. Scale effort to complexity, keep sources attached from the first finding to the last sentence, and never let a failure look like absence of evidence. **A bounded budget is a requirement, since token use is the main cost and the main driver of quality.**

> *Before: AI06 L6 reports Anthropic's published system. Teaches: a full design with the ten-part template. Used by: AI13's Scenario 3 questions.*

This is a recommended reference design, not Anthropic's or any company's system. It draws on ideas from Anthropic's post, and all sizes and costs are illustrative assumptions.

1. Scope

In scope	Out of scope for version 1
Questions needing many sources	Real-time alerts
Web and internal document search	Paywalled sources without licenses
Cited reports, with conflicts and gaps shown	Automated decisions based on reports
Effort scaled per question	Collaborative editing

Why multi-agent? The work fans out, findings exceed one context window, and paths can't be planned in advance: all the fit criteria from Anthropic's post. If your eval shows that a single agent does as well on your queries, use that.

2. Requirements

Type	Requirement	Target (engineering choice)
Quality	Citation accuracy	Sampled, 95%+ supported
Quality	Required aspects covered, or stated as gaps	100% stated
Honesty	Failed sources appear in the report	Always
Cost	Hard budget per report	$5
Latency	Simple question / complex report	Under 1 minute / under 15 minutes
Reliability	Resume after a crash	From last completed round

3. Estimation

Cost per complex report (illustrative):

Agent	Model	Input	Output	Cost
Coordinator	Opus 5.5 ($4 / $20)	60,000	4,000	$0.32
5 subagents	Sonnet 5.5 ($2 / $10)	40,000 each	3,000 each	$0.55
Citation pass	Sonnet 5.5	30,000	2,000	$0.08
Total				**$0.95**

Scale	Calculation	Result
Reports per month	Assumed	2,000
Monthly model cost	2,000 × $0.95	**$1,900**
Peak concurrent reports	Assumed	20
Peak concurrent subagents	20 × 5	**100**

Web search fees and caching aren't included. Coordinator input is resent over turns, so caching lowers it.

The budget is real protection. Each extra round multiplies spend. A spawn loop that ran 50 subagents would add $5.50 to one report, which is why the $5 cap stops it.

4. API

```
POST /v1/research
{ "question": "How is AI changing hospital operations?",
  "required_aspects": ["radiology", "triage", "billing", "nursing"],
  "max_budget_usd": 5 }
```

```
202 Accepted   {"report_id": "r_77"}

GET /v1/research/r_77/events
event: plan       {"subagents": 4, "effort": "complex"}
event: progress   {"round": 1, "gaps": ["billing", "nursing"]}
event: done       {"status": "COMPLETE_WITH_GAPS", "cost_usd": 1.12, "url": "..."}
```

Decision	Why
Asynchronous	Reports take minutes
`COMPLETE_WITH_GAPS` is its own status	A partial report isn't a failure, and it isn't full success either
Optional `required_aspects`	Gives the coverage check a target

5. Data model

```
-- Recommendation, not run.
CREATE TABLE reports (report_id text PRIMARY KEY, question text, status text,
  budget_usd numeric, spent_usd numeric, plan jsonb);
CREATE TABLE subagent_runs (run_id text PRIMARY KEY, report_id text, round int,
  aspect text, status text, attempted text, artifact_uri text);
CREATE TABLE findings (finding_id text PRIMARY KEY, report_id text, claim_key text,
  value text, excerpt text NOT NULL, source_url text NOT NULL, published date);
```

`findings` is the claim-source mapping from AI05 L4. `excerpt`, `source_url` and `published` are required, so that a finding can't exist without provenance.

6. High-level design

V1: one agent with search tools.

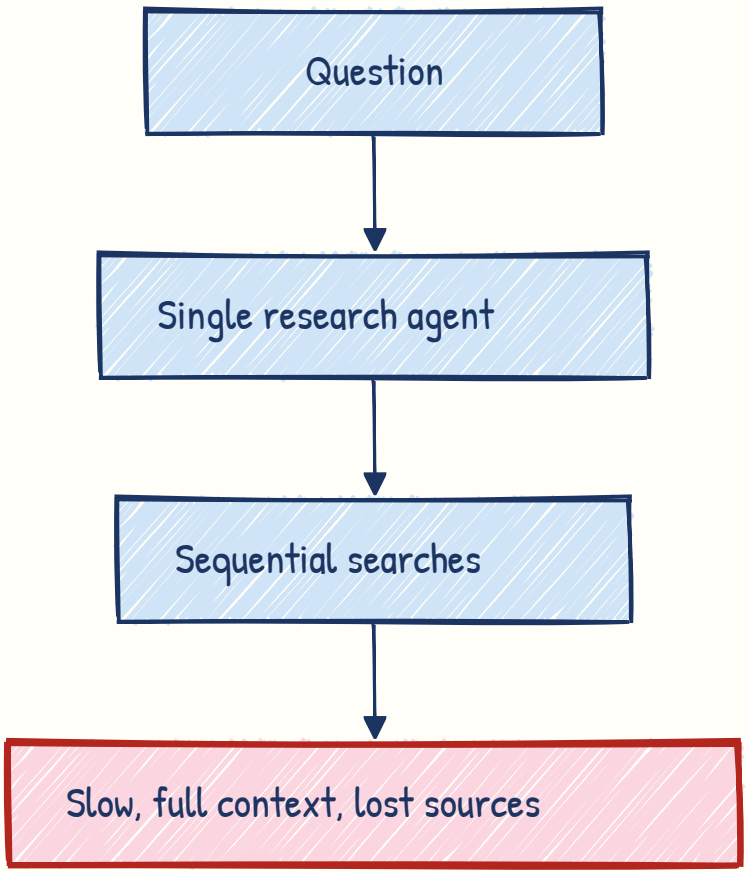


Figure 1. Baseline. A single agent searches in sequence, and its context fills with raw results.

V2: a fixed pipeline of subagents. The parts now run separately. New problem: **it always costs the same, it can't refine, and prose summaries lose sources between stages.**

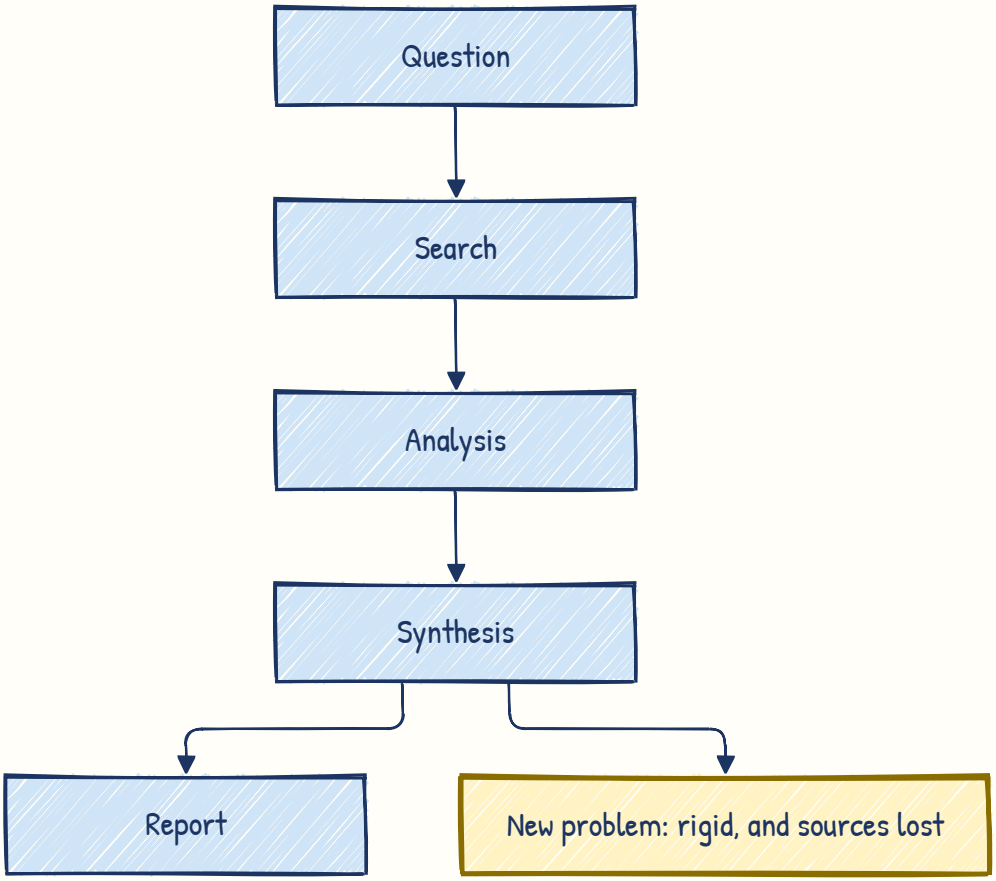


Figure 2. Improved. Stages run in isolation, but the pipeline never adapts, and attribution erodes at each step.

V3: production.

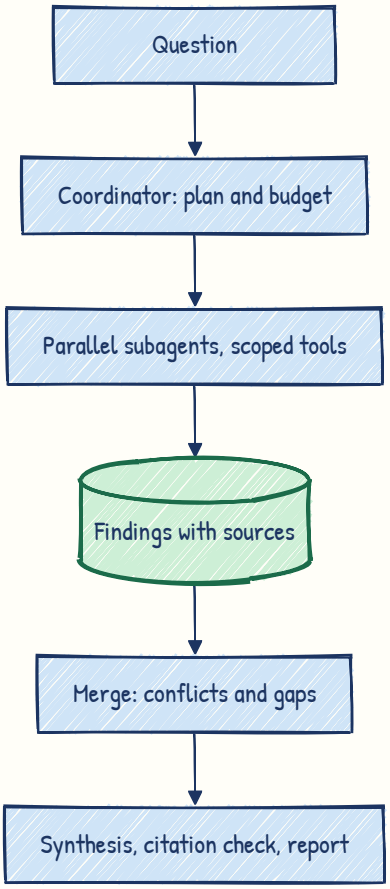


Figure 3. Production. A coordinator scales effort and fans out in parallel. Findings land in a store with their sources, conflicts are annotated in code, and a separate agent checks citations.

Component	Tools	Why it exists	How it communicates
Coordinator	Spawn tool, read findings	Plans, selects, finds gaps	Complete prompts, then references back
Search subagents	`web_search`, `load_document`	Breadth, each in its own context	Writes findings to the store
Analysis subagents	`extract_data_points`, `summarize_content`	Depth on documents	Same
Merge	None: this is code	Conflicts shouldn't depend on judgment	Reads findings, annotates
Synthesis	`verify_fact` only	Writes, without wandering	Gets the merged evidence in its prompt
Citation check	Read only	Independent review (TS 4.6)	Flags unsupported claims

Request flow

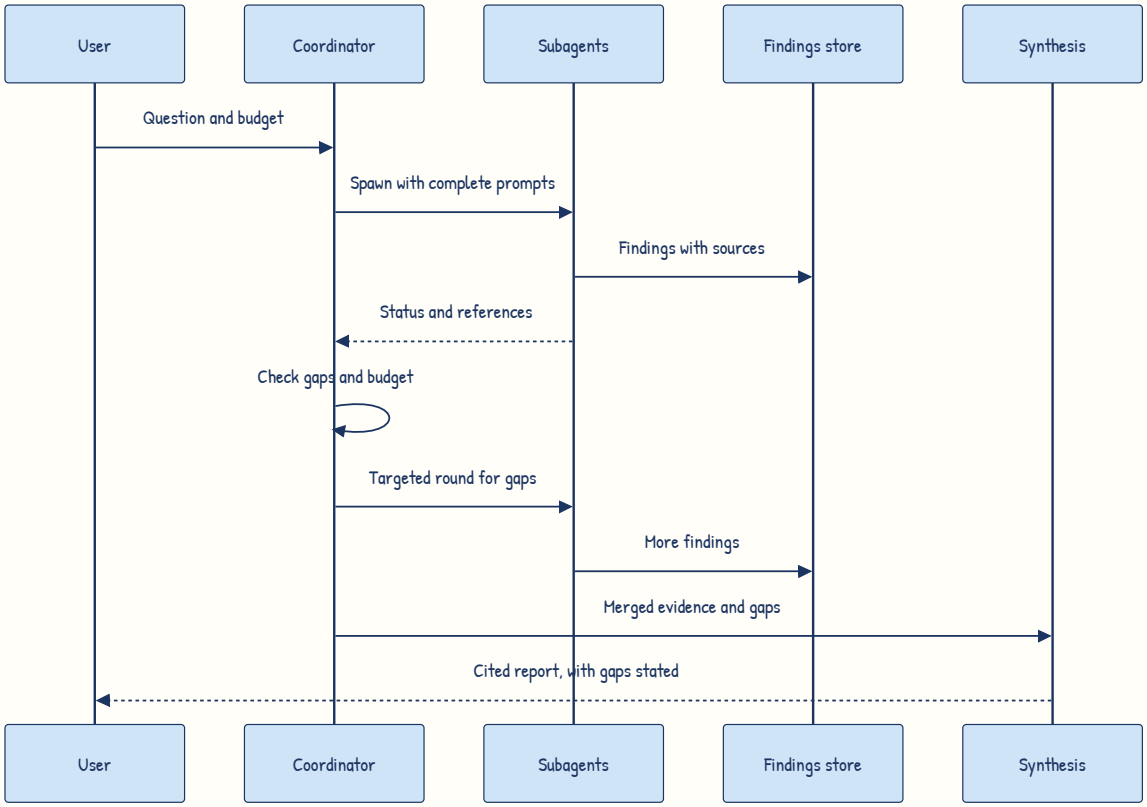


Figure 4. One report, end to end. Parallel subagents run in one step, and the gap check decides whether another round is needed.

7. Deep dives

Deep dive 1: scaling effort

Anthropic's published heuristic makes a good starting point:

Query type	Subagents	Tool calls	Example
Simple fact	1	3–10	"When was the policy updated?"
Direct comparison	2–4	10–15 each	"Compare these two vendors"
Complex research	More than 10	Divided by responsibility	"Map the whole market"

Put this in the coordinator's prompt with examples. Then **enforce the budget in code**, since the prompt only steers.

Deep dive 2: provenance through the pipeline

Reuses AI05 L4: structured findings, `DIFFERS_BY_DATE` and `CONTESTED` labels, excerpt verification.

Stage	Rule
Subagent	Must return claim, excerpt, URL and date
Merge	Code groups by claim, and labels conflicts
Synthesis	Gets conflicts as data, and shows both sides with attribution
Citation check	Independent context checks that excerpts support claims
Report	Established findings, contested findings, coverage gaps

Deep dive 3: failure is not absence

Real output from `ResearchCoordinator.java`. Subagents are simulated:

```
billing    PARTIAL  1 finding(s), tried: journal search worked; payer portal timed out twice

Coverage notes for the report:
  billing is partial: journal search worked; payer portal timed out twice
```

The subagent retried locally (AI06 L5), then passed up what it attempted, and what it found. So the report can honestly say: "Billing findings rest on one source, because the payer portal was unavailable."

Deep dive 4: evaluating research

From Anthropic's approach (AI06 L6):

Practice	Detail
Start small	About 20 real queries, and big effects will show
Judge outcomes, not paths	Three sources or ten can both be right
Rubric	Factual accuracy, citation accuracy, completeness, source quality, tool efficiency
Humans too	They caught preference for SEO content farms
Add to the set	Ask for a topic with a known failed source, and check the gap appears

8. Scaling

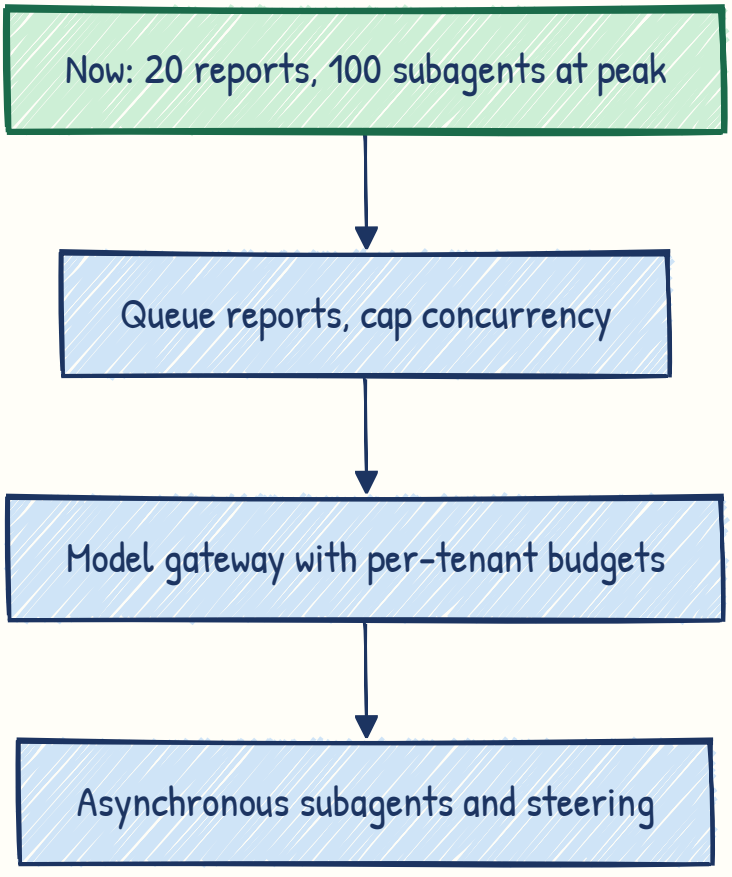


Figure 5. Scaling stages. Fan-out multiplies rate limit pressure, so concurrency control arrives early.

Limit	Signal	Response
Rate limits	429s during fan-out	Concurrency cap, backoff, fewer subagents
Cost	Spend per report	Effort scaling, budget, cheaper subagent model
Search provider	Quota errors	Multiple providers, and report gaps
Coordinator waits	Lead blocked per batch	Anthropic names this bottleneck. Async helps, but state gets harder

Failure handling

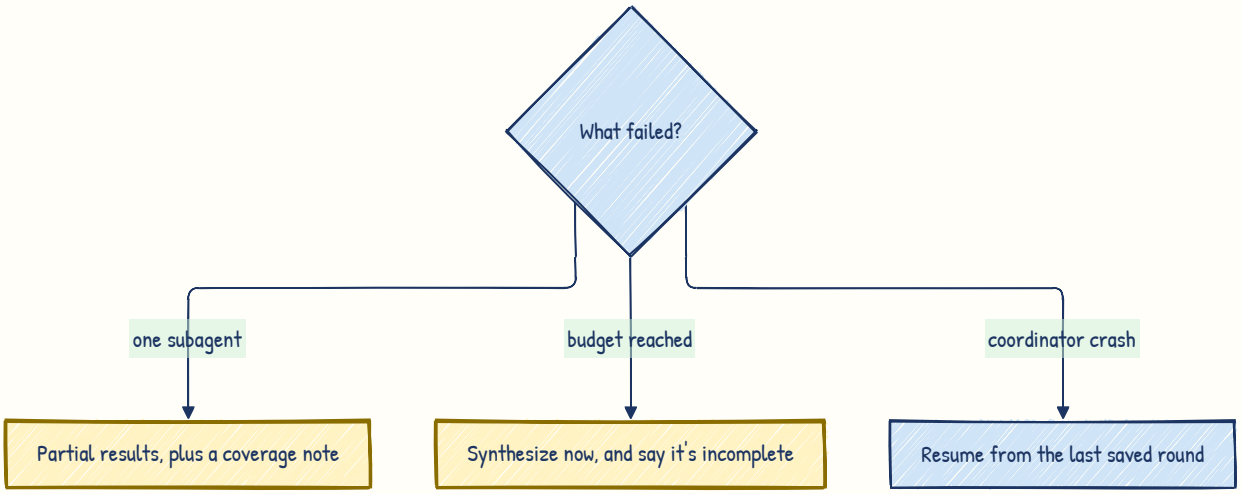


Figure 6. Failures by scope. A subagent's failure degrades a section, and a budget or coordinator failure ends the run early, with what's been found.

Failure	Detection	Containment
Spawn explosion	Spend and depth	`max_budget_usd`, depth and concurrency limits
Injection from a web page	Sampled review	Search agents have no write tools. Treat findings as data (AI09 L2)
Fabricated citation	Excerpt check	Reject in code, and re-request
Two articles repeat one source	Citation review	Group by origin, where known
All sources fail	Round produces nothing	Say so, and stop rather than generate from memory

9. Trade-offs

Decision	Chosen	Alternative	Why
Topology	Hub and spoke	Peer chat	Observability and control
Plan	Dynamic	Fixed pipeline	Open-ended work
Models	Opus 5.5 lead, Sonnet 5.5 subagents	All Opus	Anthropic's post used a similar split. Evaluate on yours
Conflict merge	Code	Synthesis decides	Don't let judgment hide conflicts
Context passing	Store with references	Everything in prompts	Fidelity, fewer tokens
Citations	Separate check	Synthesis self-checks	Independent review

Where this is overkill: questions one search can answer, or a corpus small enough for RAG with citations (AI05).

10. Follow-up questions

Q Follow-up 1. Reports read well, but a sampled check finds 15% of citations don't support their claims. Where do you look?

A First at where attribution is lost. Are subagents returning excerpts, and does synthesis receive them, or summaries? Is the citation check independent, and is it blocking? API Citations guarantee valid pointers, not support, so sampling needs to continue (AI05 L4).

Q Follow-up 2. A competitor's system answers faster with one agent. Should you drop multi-agent?

A Decide by eval. Anthropic's token and quality findings are from its tasks. Run both on your queries, comparing quality per dollar and time, by query type. A router that sends simple questions to one agent is likely the outcome.

Q Follow-up 3. Spend per report doubled after a prompt change, with no quality gain. What now?

A Compare traces for subagent count, rounds and tool calls. Small lead changes can shift subagent behavior unpredictably, as Anthropic reports. Roll back, add spend per query type to the eval, and gate on it.

Q Follow-up 4. Could subagents share a scratchpad instead of going through the coordinator?

A A shared findings store is fine and useful, since agents write artifacts and pass references. Direct messaging between agents is different. It weakens observability, consistent error handling and control, so routing decisions should stay with the coordinator.

Q Follow-up 5. Build this in Java, or the Agent SDK?

A The Agent SDK is Python and TypeScript, with subagents, sessions and hooks built in. Managed Agents offers hosted multi-agent workflows, in beta. From a Java service, you'd orchestrate through the Messages API with virtual threads, as in this module's lab, and own state and limits. Choose by team and hosting needs (AI07, AI08).

Certification lens

How this lesson maps to CCAR-F

This is Scenario 3. Watch for wrong answers that blame a downstream agent, hide failures, drop sources or pick one number. These original practice questions are not real or leaked.

Q Q1 · Exam style, scenario: Multi-Agent Research System · Medium. A simple factual question spawned 12 subagents, and cost as much as a full report. What's the best change?

A. Use a cheaper model for all subagents.

B. Have the coordinator analyze query complexity, and select only the subagents needed.

C. Limit every query to 2 subagents.

D. Remove the coordinator.

A **B** (TS 1.2). A leaves it wasteful. C harms complex queries. D removes coordination.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Hard. Synthesis output contains a gap: nothing on regulation. What should the coordinator do?

A. Tell synthesis to add what it knows about regulation.

B. Publish, as the rest is good.

C. Fail the run.

D. Re-delegate targeted queries on regulation, then re-invoke synthesis, within budget.

A D (TS 1.2). A risks fabrication. B hides the gap. C wastes the work.

Q Q3 · Exam style, scenario: Multi-Agent Research System · Medium. Which coordinator prompt best enables subagents to adapt?

A. Goals and quality criteria: what to find, format, boundaries, when to stop.

B. A numbered list of the exact searches to run.

C. "Do thorough research."

D. The full text of every tool's source code.

A A (TS 1.3). B removes adaptability. C is too vague, which Anthropic links to duplicated work. D wastes context.

Module Revision

Revision - AI06

→ Workflow: code picks the path. Agent: the model picks it. Climb only as needed.

→ Five patterns: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.

→ Loop on tool_use, finish on end_turn. Never parse text. A limit is an alarm.

→ Append the whole reply, and return all results in one user message.

→ Descriptions are the main selection signal. Fix them before adding infrastructure.

→ Scope tools by role. Forced tool_choice returns a 400 on Opus 5.5.

→ MCP: host, client, server. Tools, resources, prompts. Model, application, user.

→ isError is in the protocol. errorCategory and isRetryable are your convention.

→ Empty results are success, and failures must never look empty.

→ .mcp.json for teams, ~/.claude.json for personal, variable expansion for secrets.

→ Multi-agent costs about 15 times chat's tokens, per Anthropic.

→ Subagents inherit nothing. Parallel means several calls in one response.

→ Exam says Task, and products say Agent.

→ Sessions save the conversation, not files. Forks share files.

→ Stale results: fresh session with a summary, or name what changed.

→ Guaranteed rules go in code. Exit 1 doesn't block.

→ Escalate when asked, on a policy gap, or on no progress. Not on sentiment.

→ Handoffs stand alone.

Self-assessment

#	Without notes, can you answer?
1	Define workflow and agent, in Anthropic's terms.
2	Give the five workflow patterns, with a use for each.
3	Name the four checks before choosing an agent.
4	When do you choose a chain, and when dynamic decomposition?
5	Walk through the agentic loop's steps.
6	Give three anti-patterns for stopping a loop.
7	What should happen on `max_tokens`, `pause_turn` and `refusal`?
8	How are parallel tool results returned?
9	Compare Tool Runner, Agent SDK and Managed Agents.
10	What goes in a good tool description?
11	What's the first fix for tool misrouting, and why not a classifier?
12	How do you rename, split and consolidate tools?
13	What do `auto`, `any` and forced selection do? What's different on Opus 5.5?
14	What's poka-yoke for tools? Give Anthropic's example.
15	When would you use tool search, programmatic calling and tool examples?
16	Explain host, client and server.
17	Who controls tools, resources and prompts?
18	How do resources reduce exploratory calls?
19	Protocol error vs tool execution error?
20	Name four error categories, and what each should trigger.
21	Why is returning empty on failure so dangerous?

#	Without notes, can you answer?
22	What four things should a propagated error carry?
23	Compare the three MCP scopes. How do you keep secrets out?
24	Describe Anthropic's research architecture, with its main figures.
25	What should a subagent's prompt contain?
26	What does a subagent inherit?
27	How do you run subagents in parallel?
28	A report misses subtopics. Where do you look first?
29	When do you resume, fork or start fresh?
30	What doesn't a session or a fork protect?
31	Describe Anthropic's long-running harness.
32	What's the difference between PreToolUse and PostToolUse? Give a use for each.
33	Which exit code blocks? How do you make a hook fail closed?
34	Give three escalation triggers, and two bad ones.
35	What goes in a handoff?
36	In the support design, what does the model decide, and what does code decide?
37	In the research design, where do conflicts get handled, and why?

Where this module is used

Idea	Where it returns
Patterns and frameworks	AI07
Loop, tools and policy in Java	AI08 L3
MCP servers in Java	AI08 L4
Least privilege, injection, sandboxing	AI09
Tracing and evaluating agents	AI10
Hooks, subagents and sessions in Claude Code	AI12
Domains 1 and 2 banks, Scenarios 1 and 3	AI13

Working space